
```

maxdepth = 3
def getsplit(depth,data,variables,outcome_variable):
    --snip--
    generated_tree = [[best_var,float('-inf'),best_split,[]],
[best_var,\best_split,float('inf'),[]]]

    if depth < maxdepth:
        splitdata1=data.loc[data[best_var] <= best_split,:]
        splitdata2=data.loc[data[best_var] > best_split,:]
        if len(splitdata1.index) > 10 and len(splitdata2.index) > 10:
            generated_tree[0][3] = getsplit(depth +
1,splitdata1,variables,outcome_variable)
            generated_tree[1][3] = getsplit(depth +
1,splitdata2,variables,outcome_variable)
        else:
            depth = maxdepth + 1
            generated_tree[0][3] = best_lowermean
            generated_tree[1][3] = best_highermean
    else:
        generated_tree[0][3] = best_lowermean
        generated_tree[1][3] = best_highermean
    return(generated_tree)

```

[Listing 9-4:](#) *A function that can generate a tree of a specified depth*

In this updated function, when we define the `generated_tree` variable, we now add empty lists to it, instead of means. We insert means only in terminal nodes, but if we want a tree that has greater depth, we need to insert other branches within each branch (that's what the empty lists will contain). We also added an `if` statement with a long chunk of code at the end of the function. If the depth of the current branch is less than the maximum depth we want in a tree, this section will recursively call the `get_split()` function again to fill in another branch inside it. This process continues until the maximum depth is reached.

We can run this code to find the decision tree that leads to the lowest error in happiness predictions for our dataset:

```

variables = ['rlgdgr','hhmb','netusoft','agea','eduyrs']
outcome_variable = 'happy'
maxdepth = 2
print(getsplit(0,ess,variables,outcome_variable))

```
